PROGRAMACIÓN ORIENTADA A OBJETOS — GRUPO 07

Proyecto — Etapa 2

Simulador hospitalario: encapsulamiento, herencia, interfaces y polimorfismo

Profesor: René Adrián Dávila Pérez
Equipo 6:
323331229
426060019
323217156
312109239
323195386
321321976

INTRODUCCIÓN

De la Etapa 1 a la Etapa 2

ETAPA 1

Simulador de sistema hospitalario con clases simples: Sistema, Medico, Enfermero y Paciente. Con restricciones de asignación por especialidad y límites de pacientes.

ETAPA 2

Las mismas funciones, pero con otra arquitectura: encapsulamiento, paquetes, herencia, clases abstractas, interfaces y polimorfismo. Soporta atención general y de especialidad con estructuras de datos unificadas.

Motivación: un sistema más robusto, escalable y seguro, sin el código duplicado que apareció en la etapa 1.

INTRODUCCIÓN

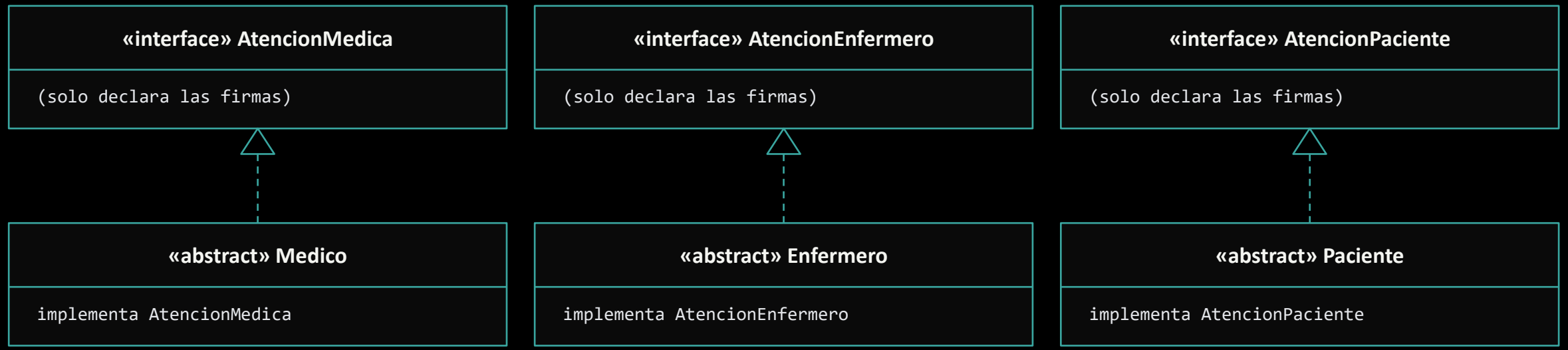
Objetivos de la etapa 2

Objetivo general: *evolucionar la arquitectura del simulador aplicando encapsulamiento, herencia, clases abstractas, interfaces y polimorfismo.*

1. Aplicar encapsulamiento con modificadores de acceso: private y protected.
2. Organizar el código con package, import y Fully Qualified Names.
3. Paciente, Medico y Enfermero como clases abstractas, con sus subclases (General, Especialista, Jefe de Enfermería).
4. Demostrar polimorfismo estático y dinámico, guardando subclases en estructuras unificadas de su clase padre.
5. Implementar la interfaz AtencionMedica y, opcionalmente, empaquetarla como archivo JAR.

DESARROLLO

Interfaces y clases abstractas



Interfaz = contrato

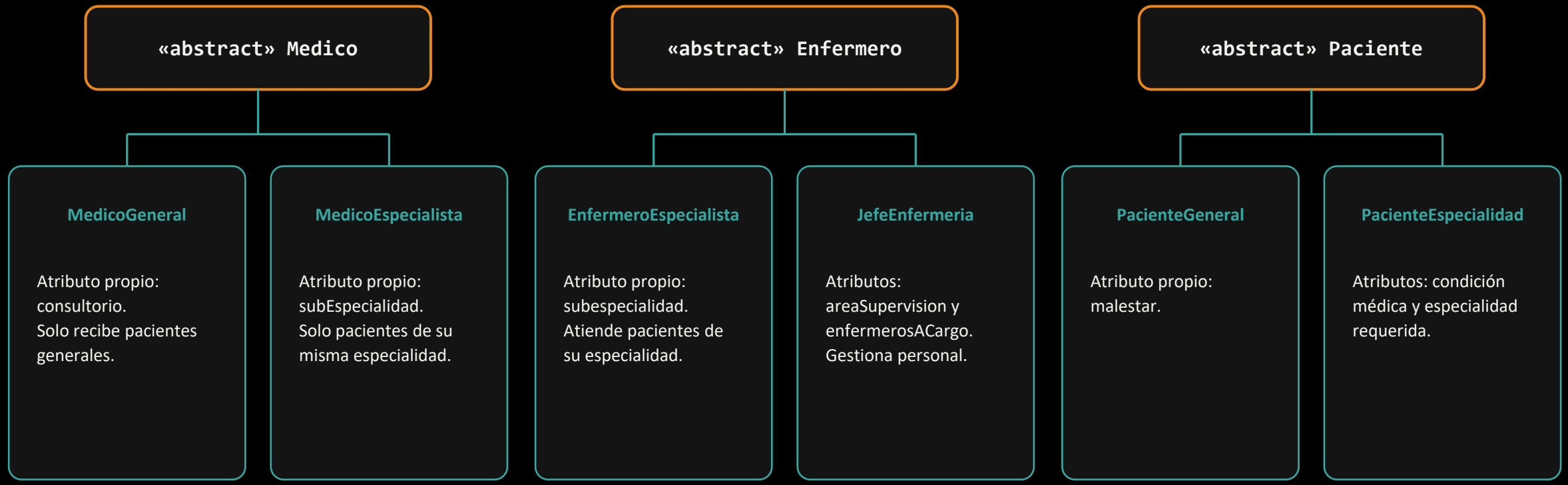
No dicta cómo el médico da consulta, solo exige que pueda hacerlo.

Clase abstracta

Ya no se puede hacer new Medico(): hay que elegir el tipo de médico.

DESARROLLO

Jerarquías de herencia



Cada subclase incorpora al menos un atributo propio y un comportamiento único.

DESARROLLO

Encapsulamiento: de private a protected

private (etapa 1)

Ni las clases hijas lo ven. Obliga a usar getters solo para leer sus propios datos heredados, algo poco eficiente.

protected (etapa 2)

Las subclases acceden directo a los atributos para validar. Hacia afuera sigue cerrado: Sistema y Main no pueden modificarlos.

Datos que no se pueden cambiar

No hay setters para cedula ni especialidad: la cédula profesional nace con el objeto y no se debe alterar. Los setters solo existen para el estado dinámico, como las banderas booleanas enConsulta del paciente.

DESARROLLO

Construcción de objetos con super

1 `new MedicoEspecialista`

El constructor recibe todos los datos requeridos e invoca de inmediato a super(...).

2 `super(...)`

Viaja a la superclase Medico: inicializa los atributos protegidos (nombre, cédula) y crea la lista de pacientes.

3 `De vuelta en la subclase`

Inicializa su atributo privado y exclusivo: subEspecialidad.

Flujo de inicialización descendente: de la superclase hacia la subclase.

DESARROLLO

Especialización y sobrescritura (@Override)

MedicoGeneral · agregarPaciente

Aplica su regla: un médico general solo recibe pacientes generales, y lo evalúa con instanceof.

MedicoEspecialista · agregarPaciente

Usa instanceof para buscar un PacienteEspecialidad y además compara con .equals() que las dos especialidades coincidan exactamente.

JefeEnfermeria

Rompe el molde: agrega areaSupervision y la lista enfermerosACargo. Su método supervisarEnfermeros() recorre a sus subordinados e imprime sus métricas.

toString() con @Override

En las clases concretas muestra el estado del objeto, con sus atributos específicos (como subEspecialidad), en lugar de un código hash.

DESARROLLO

Polimorfismo y la clase Sistema

- Sistema declara sus listas con las clases abstractas, como List<Medico>: guarda juntos a generales y especialistas.
- asignarPaciente(Paciente, Medico) recibe argumentos genéricos y actúa como filtro.
- Con instanceof «desempaca» los objetos y valida límites de pacientes y coincidencia de especialidades.
- Despacho dinámico: la JVM detecta el objeto real (por ejemplo, un MedicoEspecialista) e invoca su versión sobrescrita.

```
List<Medico> medicos;

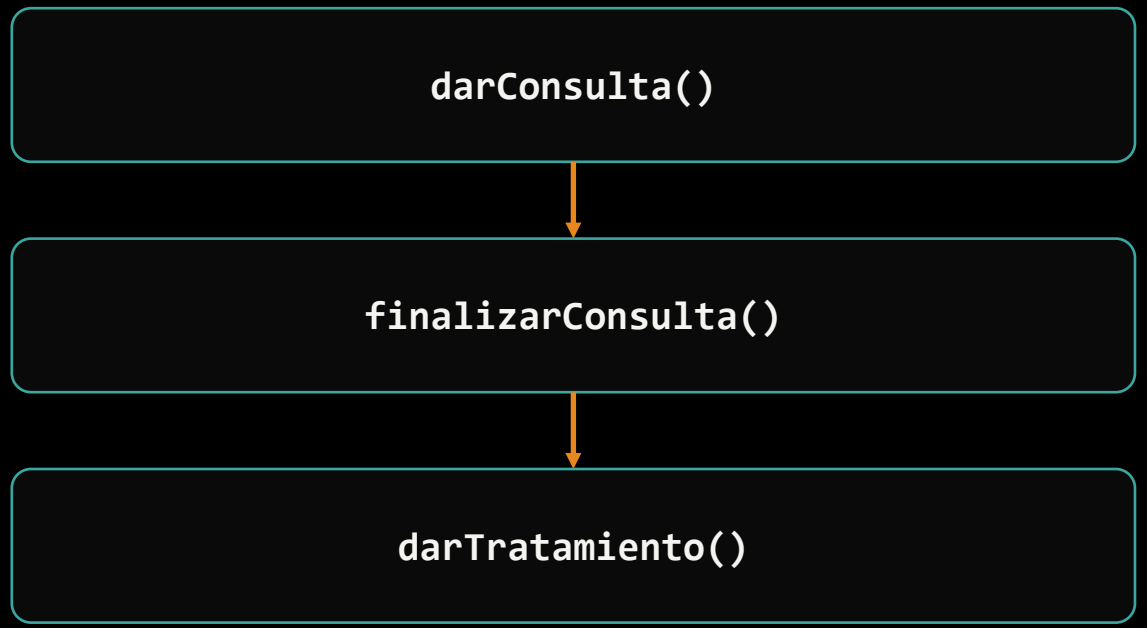
asignarPaciente(Paciente, Medico)
  // 1. instanceof: desempaca
  // 2. valida límites y especialidad
  // 3. medico.agregarPaciente(paciente)
  //    -> despacho dinámico
```

Polimorfismo dinámico: el mismo mensaje, la versión del objeto real.

DESARROLLO

Main: polimorfismo de inclusión

- Una variable de tipo Medico se instancia como MedicoEspecialista.
- Al pasarla a asignarPaciente, el sistema la usa sin saber su subtipo exacto.
- Se simula el ciclo clínico del hospital: consulta, fin de consulta y tratamiento.
- Main nunca modifica un atributo directamente: el diseño encapsulado funciona.



El médico cambia enConsulta del paciente y el enfermero lo valida antes del tratamiento.

RESULTADOS

Colecciones unificadas con polimorfismo

MÉDICOS (2)

MedicoEspecialista
Juan · cédula 12345
Cardiología · Ecocardiografía

MedicoGeneral
Pedro · cédula 67890
Consultorio 458

ENFERMEROS (2)

JefeEnfermeria
Ana · cédula 54321
Pediatría · Área: Piso 2

EnfermeroEspecialista
Luis · cédula 98765
Cardiología · Cardiología Intervencionista

PACIENTES (2)

Paciente 1
Diabetes
Requiere especialidad: Cardiología

Paciente 2
Malestar: dolor de estómago
Paciente general

Una sola colección por tipo de personal, aunque contenga subclases distintas.

RESULTADOS

Validación de asignaciones con instanceof

RECHAZADAS

- La especialidad del paciente (Cardiología) no coincide con la del enfermero (Pediatría).
- El paciente no es de especialidad.
- El médico es especialista y no puede atender pacientes generales.
- El médico no es especialista.

ACEPTADAS

- Paciente especializado → enfermero Luis.
- Paciente especializado → médico Juan.
- Paciente general → médico Pedro.

Se rechazan los cruces inválidos y se aprueban solo las coincidencias exactas.

RESULTADOS

toString() y flujo clínico heredado

- toString() muestra el estado interno y los atributos específicos de cada objeto, sin llamar a sus getters.
- Las reglas de la etapa 1 (bloqueo de pacientes en consulta o en tratamiento) se conservan en las clases abstractas.
- Todas las subclases del personal las heredan y las respetan.

```
El Dr. Juan está consultando a Paciente 1.
El paciente Paciente 1 está en consulta.
El Dr. Juan ha finalizado la consulta con
Paciente 1.
El enfermero Luis está ahora administrando
el tratamiento del paciente Paciente 1.
El Dr. Pedro está consultando a Paciente 2.
El Dr. Pedro está dando tratamiento a
Paciente 2.
```

CONCLUSIONES

De código rígido a arquitectura escalable

1

Interfaces y clases abstractas

Resolvieron la falta de estandarización: todo el personal cumple el mismo contrato.

2

Encapsulamiento y herencia

private y protected protegen los datos, super inicializa de forma segura, y la herencia elimina la redundancia (por ejemplo, el rol JefeEnfermeria).

3

Polimorfismo dinámico

Colecciones unificadas en Sistema, instanceof y despacho dinámico de métodos @Override: una arquitectura robusta y lista para escalar.

Gracias por su atención